

LedgerCrew AI — Project Documentation

A multi-agent AI SaaS back-office for Indian MSMEs — built to win a hackathon, with plenty of time to make it genuinely strong.

Note for any AI agent (Claude, Antigravity, Codex, or otherwise) picking this up: this document is the single source of truth for the project. Read it fully before writing code or design. Section 9 (Design System) has exact values — reuse them verbatim rather than inventing new colors/fonts/spacing, or the product will look inconsistent across pages built by different agents/sessions.

1. What this is

LedgerCrew AI — a crew of five specialist AI agents that runs a small business's financial back-office: reading invoices, filing GST paperwork, chasing late payments, and forecasting cash flow, so the owner gets a two-line daily brief instead of hours of admin.

Built for a hackathon, targeting: scalable, profitable, judge-appealing, fully deployable on free-tier infrastructure.

2. Core product decision & rationale

Route chosen: AI SaaS with multi-agent orchestration — not blockchain/DeFi.

Research into what wins hackathons in 2026 showed judges consistently reward projects that solve a specific problem for a specific user, use AI as a multiplier (not the whole product), and demo cleanly on the first try. “Blockchain for X” was flagged as fatigued/overengineered for most general hackathons.

A Bittensor subnet route was considered (the founder has real Bittensor/Web3 content-creation expertise) but ruled out: actual subnet building needs real compute + TAO and weeks of testnet operation, which isn't free-tier-feasible on a hackathon timeline, and the live Bittensor Subnet Ideathon cycle had already closed.

3. The idea, in detail

The pain it solves: Indian freelancers/small businesses lose hours a month to sorting invoices for GST, chasing late-paying clients, and guessing at cash flow. Real, recurring, universal — not a toy chatbot problem. Market: 60M+ Indian MSMEs, all filing GST on a recurring schedule.

Why this domain specifically: it fuses two things the founder has already built separately — billmatic's invoicing/proposal domain model, and MedPilot AI's document-OCR + multilingual + voice pipeline. Fewer novel unknowns, faster and more correct execution.

Timeline: plenty of time available — this is not a tight hackathon-weekend crunch, so the plan optimizes for depth and real validation over a longer feature list (see Section 12).

Real users: none yet as of this writing. Plan to acquire a few (dogfooding, existing X/LinkedIn audience, HackQuest community, personal network) — see Section 13.

4. Agent architecture

Agent	Role
Intake Agent	OCR-extracts vendor, amount, date, GST/HSN fields from uploaded invoices/receipts
Compliance Agent	Classifies transactions, flags GST mismatches, drafts a filing-ready GSTR-1/3B return

	(an actual draft, not just a flagged summary). Also owns tax-deadline reminders (GST filing windows, TDS due dates, advance tax installments)
Collections Agent	Drafts staged payment reminders (polite → firm → final) as invoices age
Guardrail Agent	Reviews every Collections Agent draft before it sends — catches harassment-toned or risky messaging (generator + critic pattern)
Cash-flow Agent	Forecasts 30/60/90-day cash position, flags crunch risk. Also owns payment-date reminders (e.g. payroll dates)
Orchestrator	Plans which agent runs when, holds shared state, pushes a daily “Business Brief” — reasoning is visibly streamed in the UI, not hidden

Deliberate scope decision: exactly these 5 agents + orchestrator, no more. A “financial calendar / reminder” capability was requested as a feature but assigned to the existing Compliance and Cash-flow agents rather than creating a 6th agent — reuse over proliferation.

5. Standout / differentiator features (tiered)

Tier 1 — core demo differentiators (build these)

- WhatsApp-native intake — forward an invoice photo, zero onboarding friction (the MSME target market lives on WhatsApp, not dashboards)
- Live “Agent Reasoning” panel — stream the orchestrator's plan + each agent's decisions on screen in real time
- Generator + Critic pattern (Collections Agent + Guardrail Agent) — a stronger multi-agent demo than isolated single-task agents
- GST auto-draft (not just advisory flags)

Tier 2 — the actual moat (long-term defensibility story)

- Cross-business Trust/Risk Score for clients — aggregated, anonymized payment-behavior data across tenants scores a client's reliability before you even invoice them. Needs real usage across multiple businesses to be genuine; demo with clearly-labeled seed data until real data exists — never fabricate this as real.

Tier 3 — reuses existing built expertise

- Multilingual voice interface (Hindi/Bengali/Tamil etc.) — directly reuses the OCR + multilingual + voice pattern already proven in MedPilot AI.

6. Under-the-hood architecture

- **True multi-tenancy:** Supabase Row-Level Security per business from day one
- **Per-tenant agent memory:** each business gets its own pgvector store so agents recall that business's own client history/patterns
- **Event-driven orchestration:** Postgres triggers → queue → Edge Function invocations (async, horizontally scalable)
- **Full audit log:** every agent decision logged — doubles as the data source for the Tier 1 reasoning panel
- **Retry/fallback per agent:** graceful degradation (e.g. low-confidence OCR → manual-review queue) instead of silent failure
- **Human-in-the-loop toggle:** “auto-send reminders” vs “approve before send” — real product thinking, also de-risks the collections agent

7. Security notes

- **Prompt-injection defense:** agents read untrusted uploaded documents — sanitize/validate extracted fields before they reach downstream agents; never let raw document text execute as instructions
- **RLS audit:** actually verify tenant isolation, don't just assume the policy works
- **Rate-limit fallback:** define what happens when a free-tier LLM rate limit is hit mid-use, rather than a silent failure

8. Full tech stack — verified free-tier feasibility

Layer	Tool	Free-tier status
DB/Auth/Storage/Vector	Supabase	500MB Postgres, 50K MAU, 1GB storage, pgvector included, free indefinitely (auto-pauses after 7 days inactivity)
Frontend	Next.js on Vercel	Free hobby tier
Agent orchestration	LangGraph or CrewAI	Open source
Heavy reasoning	Claude API	Existing access
Cheap/fast steps	Groq / Gemini Flash	Generous free rate limits
Demo email reminders	Resend	3,000/month, 100/day free
WhatsApp intake	Twilio WhatsApp Sandbox	Free, live in minutes, no verification
Voice transcription (STT)	Groq's free Whisper endpoint	~2,000 requests/day free
Voice reply (TTS)	Browser-native Web Speech API	Free, client-side, no API needed

Explicitly decided against Streamlit — too common at hackathons, hurts the “stand out” goal versus Next.js + Tailwind.

9. Design system (reuse exactly)

The visual concept is “ledger + crew”: a paper-ledger heritage fused with a mission-control feel for the multi-agent crew. Deliberately avoids the generic purple-SaaS-card look.

Color tokens — Light (default)

```
--bg: #F3EEDD;    --surface: #FFFDF6;    --surface-2: #EAE2C9;  
--ink: #1D1B14;  --ink-soft: #6B6555;    --line: #DCD2AE;  
--amber: #B8791E; --green: #2F6B57;    --red: #A63A2C;
```

Color tokens — Dark

```
--bg: #0F1613;    --surface: #182420;    --surface-2: #1F2E28;  
--ink: #ECE6D3;   --ink-soft: #A7A08D;    --line: #2A3733;  
--amber: #E3A63D; --green: #4FA189;    --red: #E06A54;
```

The ledger table stays paper-colored (#FFFDF6 bg, #1D1B14 text) even in dark mode — deliberate: like a lit desk in a dark room.

Color-coding legend: amber = in-progress / tax deadline / needs action; green = done / paid / on-track; red = overdue / escalation.

Typography

- **Fraunces (serif):** all display headlines, big numbers, card titles

- **IBM Plex Sans:** UI body text, labels, buttons
- **IBM Plex Mono:** tabular/numeric data only (amounts, invoice IDs, timestamps)

Motion

- **GSAP 3.12.5:** all JS-driven animation — entrance stagger, count-up numbers, chart draws, tab transitions
- **Vanta.js 0.5.24 HALO (+ three.js r134):** ambient glowing background on the login screen only; destroyed on sign-in for performance
- **Reduced motion:** all continuous animation respects prefers-reduced-motion
- **Idle motion:** crew letter-badges float+glow continuously, staggered per instance (0s/0.4s/0.8s/1.2s/1.6s)

Component vocabulary

- **Solid cards:** var(--surface) background, 1px var(--line) border, soft shadow — the default
- **Glass cards:** backdrop-filter blur, translucent — used sparingly, currently only the Financial Calendar
- **Letter-badge avatars:** 2-letter initials (In, Co, Cl, Gu, Ca) in rounded-square tiles, consistent everywhere
- **Buttons:** pill-shaped, primary = solid ink, ghost = outlined

10. Pages built so far (live links)

1. Login + main app — <https://claude.ai/artifact/6QyFimBGT4vJhghKegnh8>

Source file: ledgercrew-ui-v2.html. Login screen (Vanta halo, typed brand name, Google OAuth, WhatsApp option), Dashboard (attention queue, mini ledger, live feed, health gauge, financial calendar), Ledger table, Collections timeline, Crew agent cards with live reasoning traces.

2. Landing page — <https://claude.ai/artifact/5mKZ7kqdec7rUyPUd52pqk>

Source file: ledgercrew-landing.html. Hero, pain points, crew showcase, differentiators, stats, pricing.

11. Full page inventory (13 total)

Before login (2): Landing ✓ built • Login & sign-up ✓ built

First-run (1): Onboarding — not built

Core app (7): Dashboard ✓ • Ledger ✓ • Invoice detail — not built • Compliance/GST center — not built • Collections (list view) — partial • Cash-flow deep-dive — not built • Crew ✓ • Trust Score directory — not built

Account (2): Settings — not built • Billing & plan — not built

12. Coding agent division of labor

- **Claude:** architecture, Supabase schema + RLS, compliance/OCR reasoning prompts
- **Antigravity:** frontend/UI iteration
- **Codex:** backend glue, API routes, agent-to-agent handoff logic, tests

13. Phased build plan

- **Phase 1:** Make the core 5-agent loop bulletproof on real messy inputs — no new features yet
- **Phase 2:** Get real users (dogfood → X/LinkedIn audience → HackQuest community → personal network). If none land: don't fabricate testimonials, lean into Phases 1/3 and be upfront about pre-launch status.

- **Phase 3:** Harden — prompt-injection defense, RLS audit, rate-limit fallback, basic test suite
- **Phase 4:** Polish — onboarding flow, empty/loading states, visual identity, a 5-minute demo video
- **Phase 5:** Free-tier → scale story — a one-line answer ready for “what happens at 500 users”

Resist the urge to keep adding more agents/features with the extra time. Judges reward depth, reliability, and real impact over a longer feature list.

For an AI agent continuing this work

- Don't introduce new colors, fonts, or a 6th agent without a strong reason — check Sections 4 and 9 first.
- Any new page should reuse the same three fonts, the same CSS variable names/values, and GSAP for motion, so it matches the two artifacts already built.
- The two live artifact links above are the actual current state of the UI — read them (or their source files) before assuming what's built.
- Real users do not exist yet — never generate or present fabricated testimonials or “real” trust-score data; keep seeded data clearly labeled as such.